Державний університет «Житомирська політехніка»
Факультет інформаційно-комп'ютерних технологій (ФІКТ)
Кафедра комп'ютерної інженерії та штучного інтелекту (123)
Освітній ступінь: Баклавр (І курс)

ЗВІТ
з лабораторної роботи № 3

Дисципліна: Програмування

Тема: «Одновимірні та багатовимірні масиви, обробка символьних рядків у мові С та робота з гілками в Git»

Варіант № 2

Виконала: Веремійчук В.Є.

студентка І курсу академічної групи Кі-26-1

Перевірив:

Лисенко М. С.

Житомир — 2026

1. МЕТА

Набути практичних навичок обробки послідовних областей оперативної пам'яті: проектування алгоритмів фільтрації, пошуку екстремумів та перестановок елементів в одновимірних масивах.

Опанувати кеш-дружні алгоритми обробки двовимірних матриць з урахуванням апаратного стандарту розташування даних за рядками (Row-Major Order).

Засвоїти низькорівневу специфіку роботи з текстовою інформацією через символьні масиви типу char із кінцевим нуль-термінатором (\0).

Опанувати командну модель розгалуження в системі контролю версій Git: створення функціональних гілок (git branch , git checkout / git switch), фіксація змін та їхнє безпечне злиття в основну гілку (git merge).

2. ЗАВДАННЯ

Номер у списку: 2

Розрахунок варіанта: (2 - 1) mod 15 + 1 = 2 (Варіант 2)

2.1. Завдання 1. Обробка одновимірного масиву (task1.c)

Для Варіанта 2 реалізовано програму для обробки одновимірного масиву з 15 псевдовипадкових цілих чисел у діапазоні [-10; 10]:

- **Пошук максимуму:** Програма знаходить найбільший елемент масиву, ініціалізувавши початковий максимум значенням першого елемента arr[0].
- **Обчислення добутку між нулями:** Програма знаходить індекси першого та останнього нулів у масиві та обчислює добуток усіх елементів, розташованих між ними. Якщо в масиві менше двох нулів, виводиться повідомлення про неможливість обчислення.

lab03 > C task1.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define SIZE 15

int main(void) {
    srand((unsigned int)time(NULL));
    int arr[SIZE];

    printf("Original array:\n");
    for (int i = 0; i < SIZE; i++) {
        arr[i] = rand() % 21 - 10;
        printf("%d ", arr[i]);
    }
    printf("\n\n");

    int max_val = arr[0];
    for (int i = 1; i < SIZE; i++) {
        if (arr[i] > max_val) {
            max_val = arr[i];
        }
    }

    int first_zero = -1;
    int last_zero = -1;

    for (int i = 0; i < SIZE; i++) {
        if (arr[i] == 0) {
            if (first_zero == -1) {
                first_zero = i;
            }
            last_zero = i;
        }
    }

    printf("Maximum element: %d\n", max_val);

    if (first_zero != -1 && last_zero != -1 && first_zero < last_zero) {
        long long product = 1;
        for (int i = first_zero + 1; i < last_zero; i++) {
            product *= arr[i];
        }
        printf("Product between first (idx %d) and last (idx %d) zeros: %lld\n",
               first_zero, last_zero, product);
    } else {
        printf("Fewer than two zeros found, product cannot be calculated.\n");
    }

    return 0;
}
```

2.2. Завдання 2. Кеш-дружня обробка 2D-матриці (task2.c)

Для Варіанта 2 реалізовано програму для роботи з квадратною матрицею розміром 4x4:

- **Кеш-дружній обхід:** Вкладені цикли організовано за принципом Row-Major Order (зовнішній цикл по рядках i, внутрішній по стовпцях j) для ефективного використання кеш-пам'яті процесора.
- **Операції з матрицею:** Обчислено суму елементів, що лежать нижче головної діагоналі (де $i > j$), після чого обнулено всі елементи побічної діагоналі (за формулою $N - 1 - i$). Початковий та змінений стан матриці виведено у консоль.

lab03 > C task2.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define N 4

int main(void) {
    srand((unsigned int)time(NULL));
    int matrix[N][N];

    printf("Original matrix:\n");
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            matrix[i][j] = rand() % 50 + 1;
            printf("%6d", matrix[i][j]);
        }
        printf("\n");
    }
    printf("\n");

    int sum_below_main = 0;
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            if (i > j) {
                sum_below_main += matrix[i][j];
            }
        }
    }
    printf("Sum below main diagonal: %d\n\n", sum_below_main);

    for (int i = 0; i < N; i++) {
        matrix[i][N - 1 - i] = 0;
    }

    printf("Matrix after zeroing secondary diagonal:\n");
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            printf("%6d", matrix[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

2.3. Завдання 3. Ручна обробка С-рядків (task3.c)

Для Варіанта 2 реалізовано перевірку введеного рядка на паліндром без використання стандартних модифікуючих функцій з бібліотеки <string.h>:

- **Зчитування та сканування:** Рядок зчитується функцією fgets, після чого вручну шукається кінцевий нуль-термінатор \0 та прибирається символ переходу на новий рядок \n.
- **Перевірка паліндрому:** За допомогою двох вказівників (з початку та з кінця) порівнюються символи рядка. При цьому вручну пропускаються
- пробіли та табуляції, а великі латинські літери приводяться до малих.

lab03 > C task3.c

```c
#include <stdio.h>

#define BUFFER_SIZE 256

int main(void) {
    char str[BUFFER_SIZE];

    printf("Enter a line of text: ");
    if (fgets(str, sizeof(str), stdin) == NULL) {
        return 1;
    }

    int len = 0;
    while (str[len] != '\0') {
        if (str[len] == '\n' || str[len] == '\r') {
            str[len] = '\0';
            break;
        }
        len++;
    }

    int left = 0;
    int right = len - 1;
    int is_palindrome = 1;

    while (left < right) {
        while (left < right && (str[left] == ' ' || str[left] == '\t')) {
            left++;
        }
        while (left < right && (str[right] == ' ' || str[right] == '\t')) {
            right--;
        }

        char c1 = str[left];
        char c2 = str[right];
```

```c
        if (c1 >= 'A' && c1 <= 'Z') c1 += 32;
        if (c2 >= 'A' && c2 <= 'Z') c2 += 32;

        if (c1 != c2) {
            is_palindrome = 0;
            break;
        }

        left++;
        right--;
    }

    if (is_palindrome) {
        printf("The string IS a palindrome!\n");
    } else {
        printf("The string IS NOT a palindrome.\n");
    }

    return 0;
}
```

3. ВИСНОВКИ

Під час виконання лабораторної роботи було закріплено практичні навички обробки одновимірних та двовимірних масивів, а також символьних рядків у мові С. Опановано кеш-дружній обхід матриць у форматі Row-Major Order та покрокове сканування символьних масивів до кінцевого нуль-термінатора '\0' без використання стандартних модифікуючих функцій. Крім того, повністю відпрацьовано командну модель контролю версій Git: створення робочої гілки lab03, фіксацію поетапних змін коммітами та їхнє безпечне злиття в основну гілку main.